

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

TECNOLOGÍAS DE LA INFORMACIÓN DESARROLLO DE SOFTWARE

EJEMPLO INTEGRADOR S5

EVENTOS, ESTADOS Y CONTROL POR TECLADO

PRESENTA:

OSCAR ESPINOZA LANDETA

8 “A”

A CARGO:

ING. JOSÉ MIGUEL CARRERA PACHECO



1 Event-Driven UI: El Cerebro Detrás de la Pantalla

Una **Interfaz Orientada a Eventos** significa que el software no sigue un guion lineal. En lugar de eso, la UI es un sistema **reactivo**. Imagina que tu app es una fiesta: el código no está haciendo nada hasta que alguien "toca el timbre" (un clic, una tecla, un mensaje de la API). Ese timbre es el **Evento**.

El Ciclo de Vida del Evento (El viaje épico)

Esto es lo que la mayoría confunde. Cuando tú haces clic en un botón, el evento no aparece de la nada en el botón. Pasa por tres fases obligatorias:

1. **Capturing Phase (Fase de Captura):** El evento baja desde el `window`, pasa por el `document`, luego por el `<body>`, y va bajando por cada padre hasta llegar al elemento donde hiciste clic. Es como un paracaidista buscando su zona de aterrizaje.
2. **Target Phase (Fase de Destino):** El evento llega al elemento exacto (el `event.target`). Aquí es donde se disparan los listeners que pusiste ahí.
3. **Bubbling Phase (Fase de Burbujeo):** ¡El evento rebota! Ahora sube desde el botón hacia arriba hasta llegar de nuevo al `window`. **Dato pro:** El 99% de las veces trabajamos aquí. Si usas `e.stopPropagation()`, matas el burbujeo y los padres no se enteran de lo que pasó.

Diferencia: Imperativo vs. Basado en Eventos

- **Imperativo (Modo "Controlador"):** Tú le dices a la computadora: *"Busca el botón, cámbiale el color a rojo, ahora abre el modal"*. Si algo cambia en el HTML, tu código truena.
- **Basado en Eventos (Modo "Observador"):** Tú dices: *"Si el estado 'isOpen' cambia a true, que la UI se pinte sola"*. El código reacciona a cambios de estado disparados por eventos. Es mucho más limpio.

2 Eventos de Teclado: El Manual del Usuario Pro

Aquí es donde se nota si alguien sabe programar para humanos o solo para el mouse.

El Trío Dinámico de Eventos

- **keydown:** Es el rey. Se dispara en cuanto la tecla toca fondo. Se usa para casi todo porque detecta teclas que no escriben nada (como **Ctrl**, **Shift** o **Esc**). Además, si dejas la tecla presionada, se repite automáticamente.
- **keyup:** Se dispara cuando sueltas la tecla. Es perfecto para cuando quieres validar algo *después* de que el usuario terminó de escribir (como un campo de búsqueda).

- **keypress**: ¡Muerte a este evento! Está **obsoleto**. No detectaba bien teclas especiales y los navegadores modernos ya ni lo pelan. Ni lo menciones en el examen a menos que sea para decir que es viejo.

event.key vs event.code (No son lo mismo, ¡ojo!)

- **event.key**: Te da el valor "humano". Si presionas la tecla "a", te da "a". Si presionas "Shift + a", te da "A". Es ideal para lógica de escritura.
- **event.code**: Te da la ubicación física del botón. No importa si tu teclado está en ruso o español, la tecla física que está a la derecha del Tab siempre será "KeyQ". Se usa para juegos (WASD) o shortcuts que no deben cambiar con el idioma.

El superpoder de preventDefault()

Por defecto, si presionas "Espacio", la página hace scroll hacia abajo. Si estás en un juego o un menú, eso es un error. Usamos `e.preventDefault()` para decirle al navegador: *"Tranquilo, yo me encargo de esta tecla, tú no hagas nada"*.

3 Estados Visibles y Accesibles (El Alma de la Inclusión)

La **Accesibilidad (A11y)** no es un "extra", es parte del core. Si una persona ciega o que no puede usar las manos entra a tu web, debe poder usarla igual que nosotros.

Estados de UI: La Comunicación No Verbal

- **Focus**: Es cuando un elemento está seleccionado por el teclado (con **Tab**). Es el "cursor" de los usuarios de teclado. **Regla WCAG**: El foco *siempre* debe ser visible. Nunca pongas `outline: none` sin una alternativa clara.
- **Aria-expanded**: Un atributo que le grita al lector de pantalla: *"¡Ey! El menú que tengo aquí abajo está abierto/cerrado"*.
- **Aria-controls**: Es el "vínculo" invisible. Dice: *"Yo soy el botón que manda sobre aquel `div` de allá"*.

Gestión de Estados en React + ARIA

En React, el estado (`useState`) debe estar sincronizado con los atributos ARIA. Si cambias un `isOpen` a `true`, el HTML debe cambiar automáticamente a `aria-expanded="true"`.

Cumplimiento WCAG (Nivel Básico):

1. **Navegabilidad**: Todo debe ser alcanzable con la tecla **Tab**.
2. **Operabilidad**: Si se le puede dar clic, se le puede dar **Enter**.

3. **Contraste:** Los colores deben resaltar. Un texto gris claro sobre fondo blanco es un pecado capital en accesibilidad.

La Implementación Maestra: Menú Accesible

Para que veas cómo se junta todo, checa este componente mental que vamos a armar:

JavaScript

```
// Imagina esto en tu componente de React
const handleKeyDown = (e) => {
  // 1. Controlamos la navegación por flechas (Patrón ARIA)
  if (e.key === 'ArrowDown') {
    e.preventDefault();
    moveFocusToNextItem();
  }

  // 2. Cerramos con Escape (Feedback instantáneo)
  if (e.key === 'Escape') {
    setIsOpen(false);
    buttonRef.current.focus(); // ¡Importante! Devuelve el foco al botón
  }
};
```